AI 2002
Artificial Intelligence
Lecture 19

Mahzaib Younas

Lecturer, Department of Computer Science

FAST NUCES CFD

Perceptron Training Rule

- The ***perceptron training rule,*** which revises the weight w_i associated with input x_i according to the rule:

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

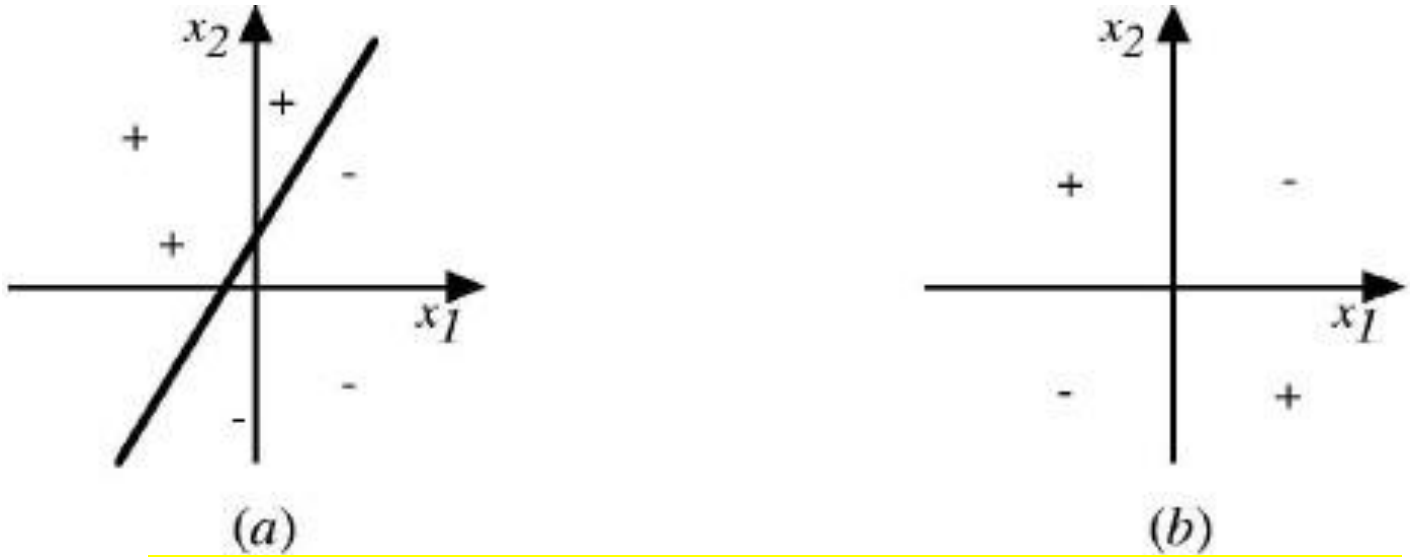
Where:

- t is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called *learning rate*

Perceptron Training Rules

- The **perceptron rule** finds a successful weight vector when the training examples are **linearly separable**,
- It fails to converge if the examples are **not linearly separable**.
- The solution is … **Delta Rule** also known as **(Widrow- Hoff Rule)**
- **Delta Rule**
- use ***gradient descent*** to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples.

Perceptron Training Rule



The **decision surface** represented by a **two-input perceptron** x_1 and x_2.

(a) A set of training examples and the decision surface of a perceptron that classifies them correctly.

(b) A set of training examples that is not linearly separable.

Delta Rule

- In **perceptron training rule** we employ ***thresholded perceptron***

$$o(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}.\mathbf{x} > 0 \\ -1 & \text{otherwise} \end{cases}$$

- The **delta training rule** is the task of training an ***unthresholded perceptron***;
  - a ***linear unit*** for which the output o is given by

$$o(\mathbf{x}) = \mathbf{w}.\mathbf{x}$$

  - A linear unit corresponds to the *first stage* of a perceptron, ***without*** the threshold.

Delta Rule

- In order to derive ***a weight learning rule for linear units***,
  - Specify a measure for the ***training error*** of a hypothesis (weight vector), relative to the training examples.

$$E(\vec{w}) \equiv \frac{1}{2}\sum_{d \in D}(t_d - o_d)^2$$

  - D is the set of training examples,
  - t_d is the target output for training example d,
  - o_d is the output of the linear unit for training example d
  - E is characterized as a function of $\boldsymbol{w}$, because the linear unit output
- o depends on this weight vector.

Phase of Delta rule

- The delta rule consist of three phases

1. Feed forward Phase
2. Error Phase
3. Update weight

Feed Forward Phase

- The perceptron computes the weighted sum of the inputs plus the bias:

$$yp = w1 \cdot x1 + w2 \cdot x2 + b$$

Compute Error/Update weight

- The error is the difference between the target output (y_e) and the predicted output (y_p):

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

- Update the weights using delta rule

Example:

- Consider the following
  - Input vector: x=[2,1]
  - Target output: t=4
  - initial weight vector: w=[0.5,−1]
  - Learning rate: η=0.1

Step 1: compute output & error

- Compute output

$$O(w.x) = (0.5)(2)+(-1)(1)$$
$$=1.0-1$$
$$=0$$

- Compute error

$$E(\mathbf{w}) = \frac{1}{2}(t-o)^2$$

$$= \frac{1}{2}(4-0)^2 = \frac{1}{2}(16) = 8$$

Step 2: Update weights

- Formula to update the weight

$$w_i := w_i + \eta(t - o)x_i$$

- Weight for w1

$$w_1 = 0.5 + 0.1(4 - 0)(2) = 0.5 + 0.8 = 1.3$$

- Weight for w2

$$w_2 = -1 + 0.1(4 - 0)(1) = -1 + 0.4 = -0.6$$

Gradient Descent

- Gradient descent search determines *a weight vector that minimizes* ***E*** by
  - Starting with an arbitrary initial weight vector,
  - Repeatedly modifying it in small steps.
  - At each step, the ***weight vector is altered in the direction*** that produces the **steepest descent** along the error surface,
  - This process continues until the **global minimum error** is reached.

Gradient Descent

- ***How can we calculate the direction of steepest descent along the error surface?***
  - This direction can be found by computing the derivative of E with respect to each component of the vector $\vec{w}$.

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{\partial w_n} \right]$$

- Notice $\nabla E(w)$ is itself a vector, whose components are the partial derivatives of E with respect to each of the w_i.

Gradient Descent

- The **gradient specifies the direction** that produces the steepest increase in E. The training rule for gradient descent is,

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

- where,

$$\Delta\vec{w} = -\eta\nabla E(\vec{w})$$

- The negative sign is present because we want to move the weight vector in the direction that ***decreases*** E.

Gradient Descent

- This training rule can also be written in its **component form**,

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w} \qquad\qquad w_i \leftarrow w_i + \Delta w_i$$

- where,

$$\Delta\vec{w} = -\eta \nabla E(\vec{w}) \qquad\qquad \Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

- The steepest descent is achieved by altering each $\partial E/\partial$w

Gradient Descent

- The vector of derivatives $\frac{\partial}{\partial w_i}$ that are from the gradient can be obtained by differentiating E from delta rule
-

$$E(\vec{w}) \equiv \frac{1}{2}\sum_{d \in D}(t_d - o_d)^2$$

$$o(\mathbf{x}) = \mathbf{w}.\mathbf{x}$$

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_{d \in D}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d \in D}\frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d \in D}2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_{d \in D}(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w} \cdot \vec{x}_d) \\
\frac{\partial E}{\partial w_i} &= \sum_{d \in D}(t_d - o_d)(-x_{id})
\end{aligned}$$

Gradient Descent

The vector of derivatives $\frac{\partial}{\partial w_i}$ that are used in term of

- The linear input x_{id}
- outputs o_d,
- target values t_d associated with the training examples

- The weight update rule for gradient descent becomes

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\, x_{id}$$

Gradient Descent

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\, x_{id}$$

Gradient Descent

- Gradient descent is an important general paradigm for learning.
- It is a *strategy for searching through a* ***large or infinite***
- *hypothesis space* that can be applied whenever
  1) the hypothesis space contains **continuously parameterized hypotheses** (e.g., the weights in a linear unit),
  2) the **error can be differentiated** with respect to these hypothesis parameters

Gradient Descent

- The **key practical difficulties** in applying gradient descent are:
  - ***converging to a local minimum*** can sometimes be **quite slow** (i.e., it can require many thousands of gradient descent steps),
  - ***if there are multiple local minima*** in the error surface, then there is no guarantee that the procedure will find the global minimum.

Gradient Descent Example:

- Lets Take an example:
- Learning rate η=0.5
- Initial weights:$w0$=−0.3 (bias)
- $w1$=0.5, w 2 =0.5
- Input vector: $\vec{x}$=[1,$x1$,$x2$] (bias input is always 1)

x1	x2	t
1	0	1
1	1	0
0	0	0
0	1	1

Solution:

- **x=[1,1,0],t=1**

$$o=-0.3+0.5(1)+0.5(0)=0.2$$

$$(t-o)=0.8$$

$$\Delta w0=0.5(0.8)(1)=0.4$$

$$\Delta w1=0.5(0.8)(1)=0.4$$

$$\Delta w2=0.5(0.8)(0)=0$$

Solution:

- **x=[1,1,1],t=0**
- o=−0.3+0.5+0.5=0.7
- (t−o)=−0.7

$$\Delta w0=0.5(-0.7)(1)=-0.35$$

$$=0.4-0.35=0.05$$

$$\Delta w1=0.5(-0.7)(1)=-0.35$$

$$=0.4-0.35=0.05$$

$$\Delta w2=0.5(-0.7)(1)=-0.35$$

$$=0-0.35=-0.35$$

Solution:

- **x=[1,0,0],t=0**
- o=−0.3(t−o)=0.3

$$\Delta w0=0.5(0.3)(1)=0.15$$

$$=0.05+0.15=0.2$$

$$\Delta w1=0.5(0.3)(0)=0$$

$$=0+0.05 = 0.05$$

$$\Delta w2=0.5(0.3)(0)=0$$

$$0+(-0.35) = -0.35$$

Solution:

- **x=[1,0,1],t=1**
- o=−0.3+0+0.5=0.2
- (t−o)=0.8

$$\Delta w0=0.5(0.8)(1)=0.4$$

$$=0.2+0.4=0.6$$

$$\Delta w1=0.5(0.8)(0)=0$$

$$=0+0.05$$

$$\Delta w2=0.5(0.8)(1)=0.4$$

$$=-0.35+0.4=0.05$$

Solution

- $w0 = -0.3 + 0.6 = 0.3$
- $w1 = 0.5 + 0.05 = 0.55$
- $w2 = 0.5 + 0.05 = 0.55$

Stochastic Gradient Descent

- The idea behind stochastic gradient descent is to approximate the gradient descent search by **updating weights incrementally**, following the calculation of the error for ***each* individual example**.

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\, x_{id}$$

$$\Delta w_i = \eta (t - o)\, x_i$$

Stochastic Gradient Descent

- One way to view this stochastic gradient descent is to consider a **distinct error function** defined for ***each*** *individual training example* ***d*** *as follows.*

$$E_d(\vec{w}) \equiv \frac{1}{2}(t_d - o_d)^2$$

  - where t, and o_d are the target value and the unit output value for training example d.
  - Stochastic gradient descent iterates over the training
- examples d in D,
  - at each iteration altering weights according to the gradient

Stochastic Gradient Descent

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each w_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \eta(t - o)x_i$$

Stochastic Gradient Descent Example:

- Lets Take an example:
- Learning rate η=0.5
- Initial weights:$w0$=−0.3 (bias)
- $w1$=0.5, w 2 =0.5
- Input vector: $\vec{x}$=[1,$x1$,$x2$] (bias input is always 1)

x1	x2	t
1	0	1
1	1	0
0	0	0
0	1	1

Step 1:

- **x=[1,1,0],t=1x = [1, 1, 0], t = 1**

$$o=-0.3+0.5(1)+0.5(0)=0.2$$

$$(t-o)=0.8$$

- Weights:

$$\Delta wi=0.5(0.8)xi$$

$$\Delta w0=0.5(0.8)(1) = 0.4$$

$$\Delta w1=0.5(0.8)(1) = 0.4$$

$$\Delta w2=0.5(0.8)(0) = 0$$

Update weights

- $w0 = -0.3 + 0.4 = 0.1$
- $w1 = 0.5 + 0.4 = 0.9$
- $w2 = 0 + 0.5 = 0.5$

- Now updated weights are

$$[0.1, 0.9, 0.5]$$

Step 2:

- **x=[1,1,1],t=0**

$$o=0.1+0.9+0.5=1.5$$

$$(t-o)=-1.5$$

- $\Delta w0= (0.5)(0-1.5)(1) = -0.75$
- $\Delta w1= (0.5)(0-1.5)(1) = -0.75$
- $\Delta w2= (0.5)(0-1.5)(1) = -0.75$
- Previous Weights:

[0.1, 0.9, 0.5]

Updated weights:

- $w0 = 0.1 - 0.75 = -0.65$
- $w1 = 0.9 - 0.75 = 0.15$
- $w2 = 0.5 - 0.75 = -0.25$

[-0.65, 0.15, -0.25]

Step 3:

- x=[1,0,0], t=0

[-0.65, 0.15, -0.25]

(1 * -0.65) + (0*0.15) +(0 * -0.25) = -0.65 + 0 + 0 = -0.65

- Error:

=(0 – (-0.65))

=0.65

Updated Weights

- $\Delta w0 = (0.5)(0.65)(1) = 0.325$
- $\Delta w1 = (0.5)(0.65)(0) = 0$
- $\Delta w2 = (0.5)(0.65)(0) = 0$

- Previous weight:
- [-0.65, 0.15, -0.25]

$$W0 = -0.65 + 0.325 = -0.325$$

$$w1 = 0.15 + 0 = 0.15$$

$$W2 = -0.25 + 0 = -0.25$$

Step 4:

- x=[1,0,1],t=1 [-0.325, 0.15, -0.25]
  - (1*-0.325) + (0*0.15) + (1*-0.25) = -0.575

- Error:

$$= t - o$$

$$=1 -(-0.575)$$

$$=1.575$$

Update weights:

- $\Delta w0 = (0.5)(1.575)(1) = 0.7875$
- $\Delta w1 = (0.5)(1.575)(0) = 0$
- $\Delta w2 = (0.5)(1.575)(1) = 0.7875$

- Update weights:

$$w0 = -0.325 + 0.7875 = 0.4625$$

$$W1 = 0.15 + 0 = 0.15$$

$$w2 = -0.25 + 0.7875 = 0.5375$$

The Key Difference

- In **stochastic gradient descent,** weights are updated upon examining ***each*** training example.
- Whereas in **standard gradient descent**, the error is summed over ***all*** examples before updating weights,
  - **Standard gradient descent** requires **more computation** per weight update step.
  - **Standard gradient descent** is often used with a **larger step size** per weight update than stochastic gradient descent.

Training rules

- **Perceptron Training Rule:** guarantee to succeed if
  - training examples are linearly separable
  - Sufficiently small learning rate

- **Delta Rule:**
  - use gradient descent
  - converges only asymptotically toward the minimum error hypothesis,
  - converges regardless of whether the training data are linearly separable.

Difference between gradient and stochastic gradient Descent Algorithm

Gradient Descent	**Stochastic Gradient Descent**
Computes gradient using the whole Training sample	Computes gradient using a single Training sample
Slow and computationally expensive algorithm	Faster and less computationally expensive than Batch GD
Not suggested for huge training samples.	Can be used for large training samples.
Deterministic on nature	Stochastic in nature